

COMPUTACIÓN DE ALTAS PRESTACIONES (CAP)

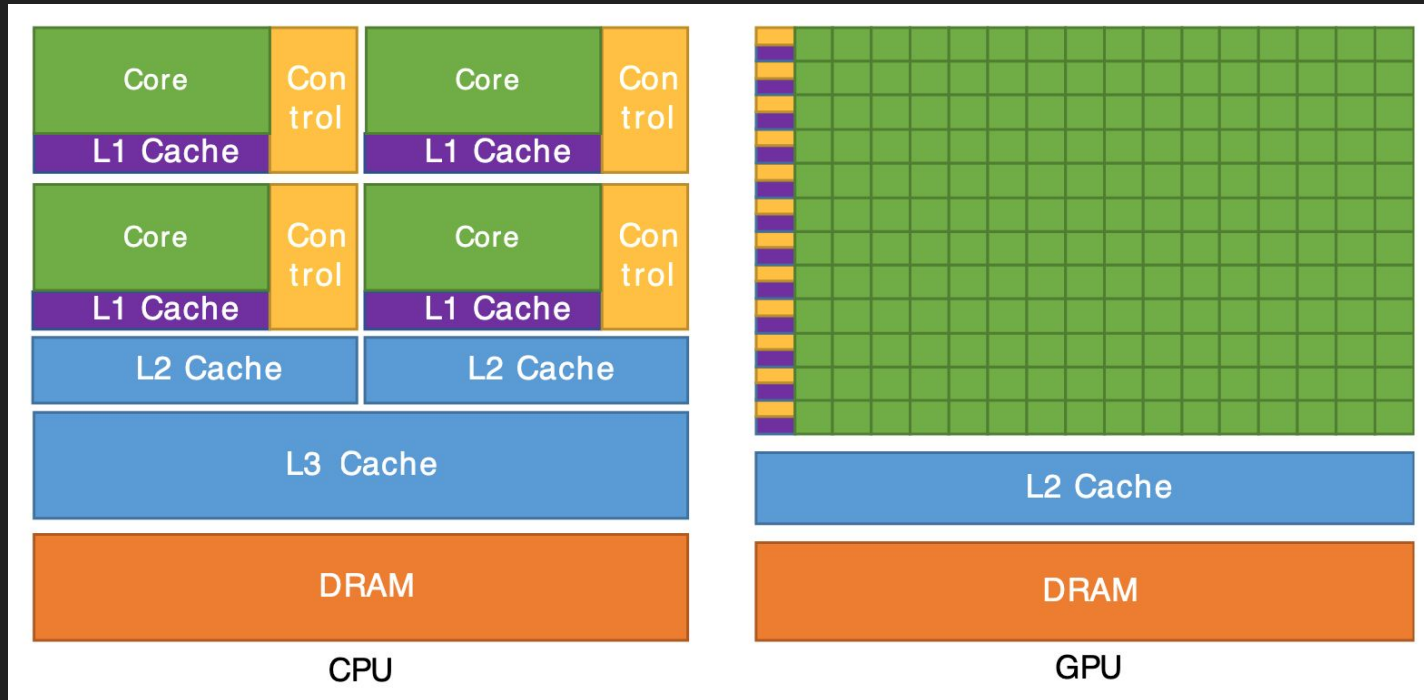
Tema 4: CUDA

Dr. Héctor Martínez Pérez
el2mapeh@uco.es

Dpto. Ingeniería Electrónica y de Computadores
Universidad de Córdoba



Arquitectura CPU vs GPU



Rendimiento GPU vs CPU

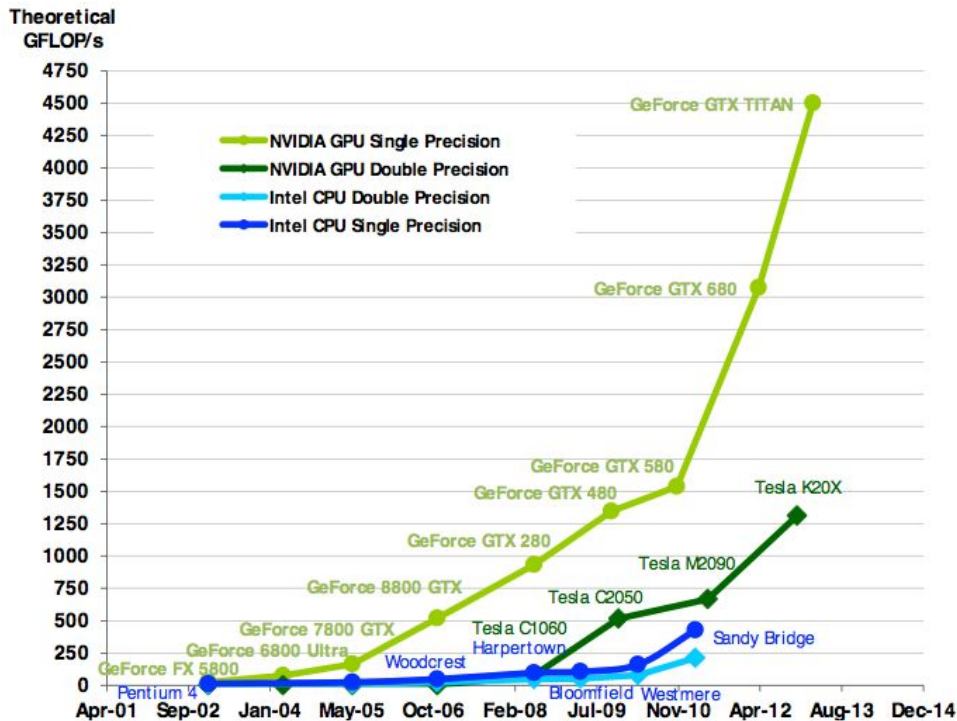


Figure 1 Floating-Point Operations per Second for the CPU and GPU

Compute Unified Device Architecture

- Dispositivo unificado para cómputo y gráficos
- Lenguaje C con extensiones propias
- Único código que se ejecuta por miles de hilos (SIMD)
- Escalable
- No todo el código es paralelizable



@DANNY_LOGO



CUDA: Tipos de Dispositivos

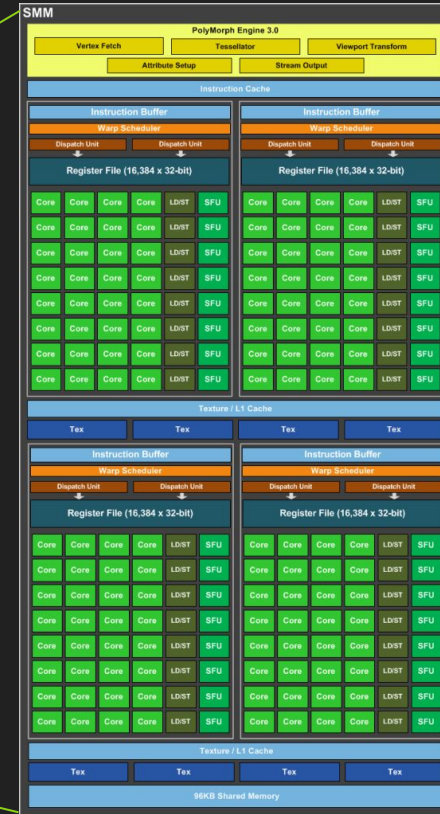
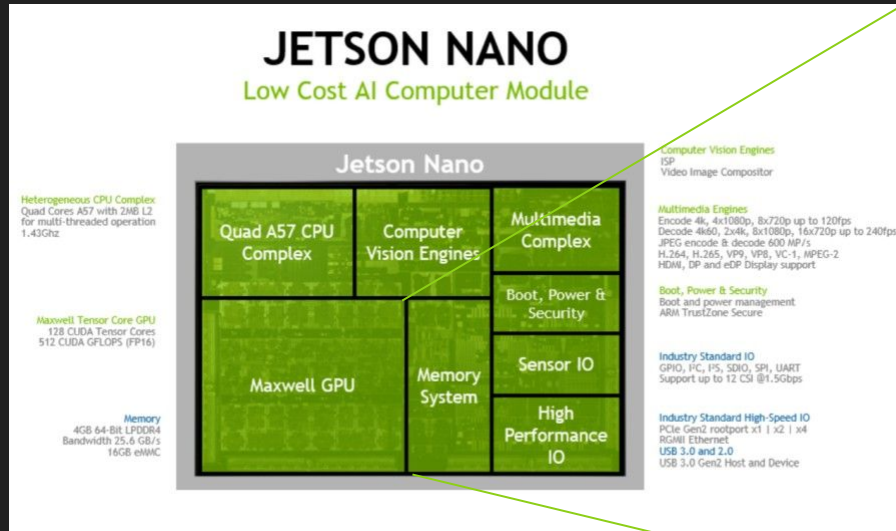
Para trabajar con CUDA necesitamos una GPU CUDA-Enable (GPU Nvidia)

Tipos de GPU Nvidia:

- **GForce:** GPU orientada a juegos y consumo general
- **Quadro:** GPU orientada a drivers optimizados para herramientas CAD y modelos 3D
- **Tegra:** sistemas heterogéneos CPU+GPU orientadas al bajo consumo y sistemas embebidos. Robótica y sistemas móviles
- **Tesla:** GPUs destinadas a cómputo. No poseen salida de vídeo



CUDA: Arquitectura Jetson Nano

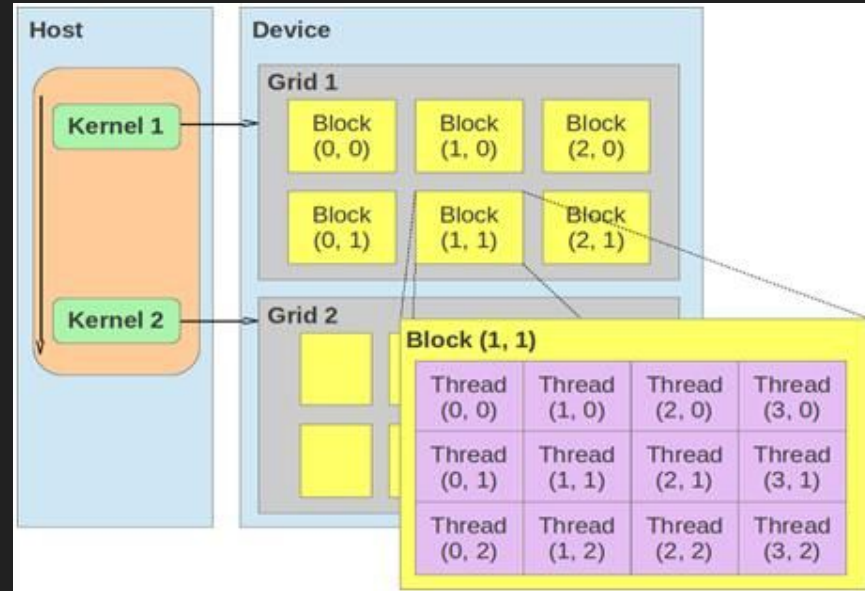


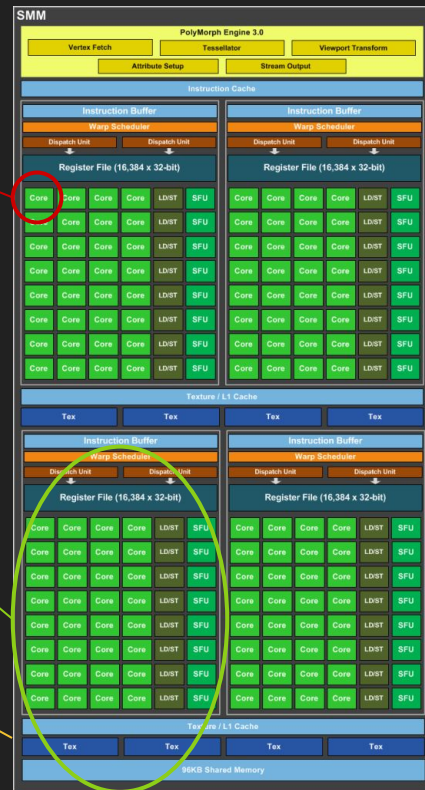
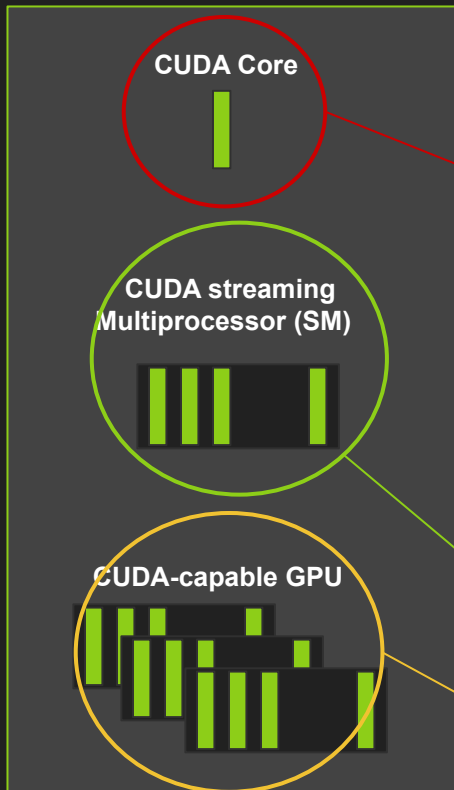
KERNEL: Función que se ejecuta en la GPU

CUDA Threads: Unidad básica de trabajo

CUDA Blocks: Conjunto de CUDA Threads

CUDA Grid: Conjunto de CUDA Blocks



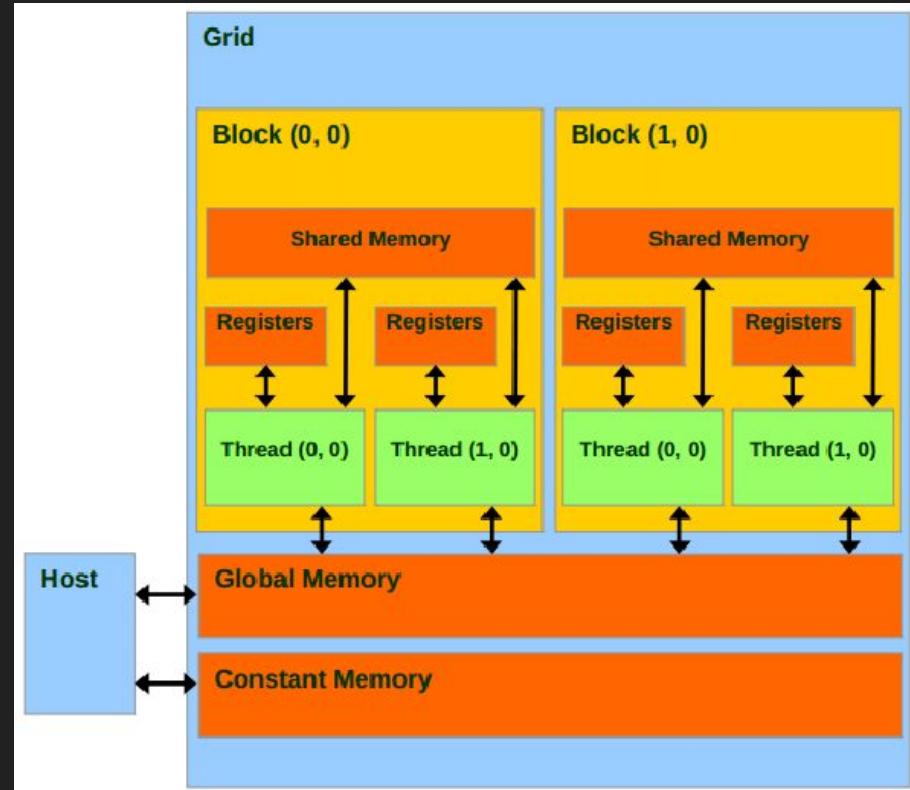


Dr. Héctor Martínez Pérez

CUDA: Jerarquía de memoria

Registros

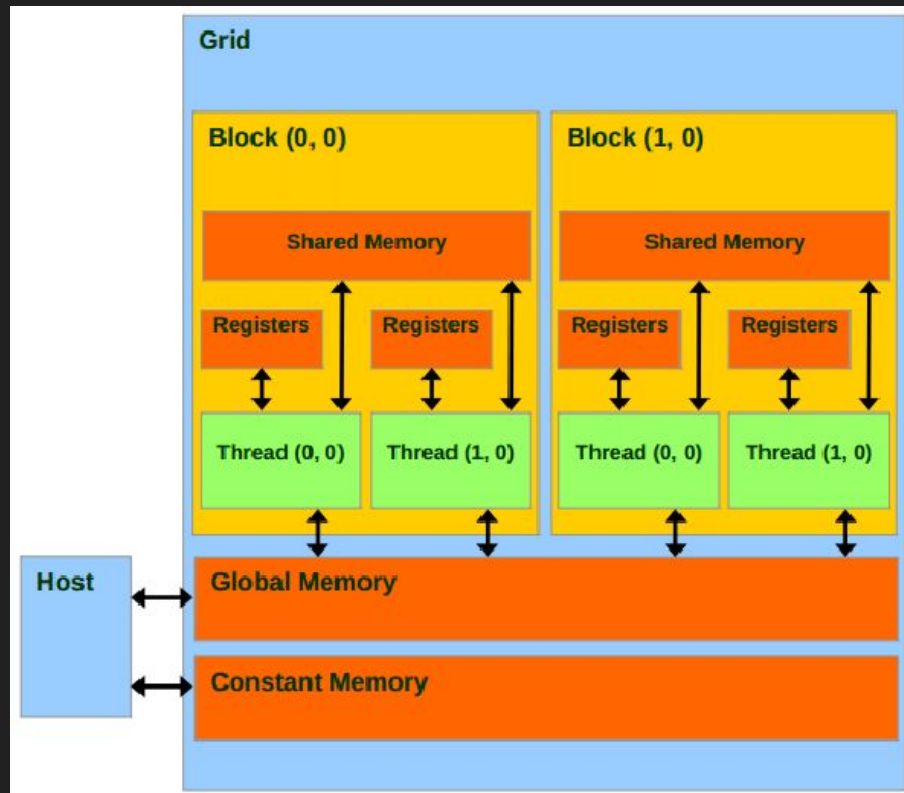
Son privados para cada hilo. Los registros asignados a un hilo no son visibles para otros hilos. Son muy rápidos en acceso, pero tienen limitaciones de tamaño



CUDA: Jerarquía de memoria

Caché L1 ó Shared Memory (SMEM)

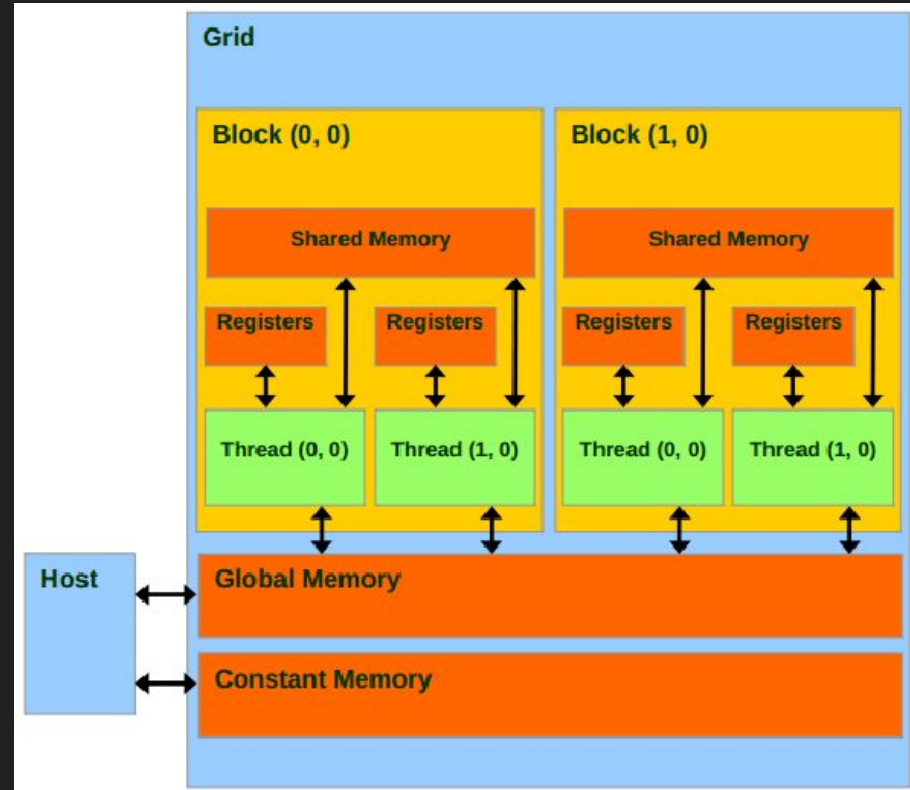
Cada SM tiene una memoria on-chip que se puede usar como memoria caché L1 y memoria compartida. Se comparte por todos los hilos en un bloque de CUDA. Al igual que los registros es una memoria muy rápida con limitaciones de tamaño.



CUDA: Jerarquía de memoria

Memoria de solo-lectura

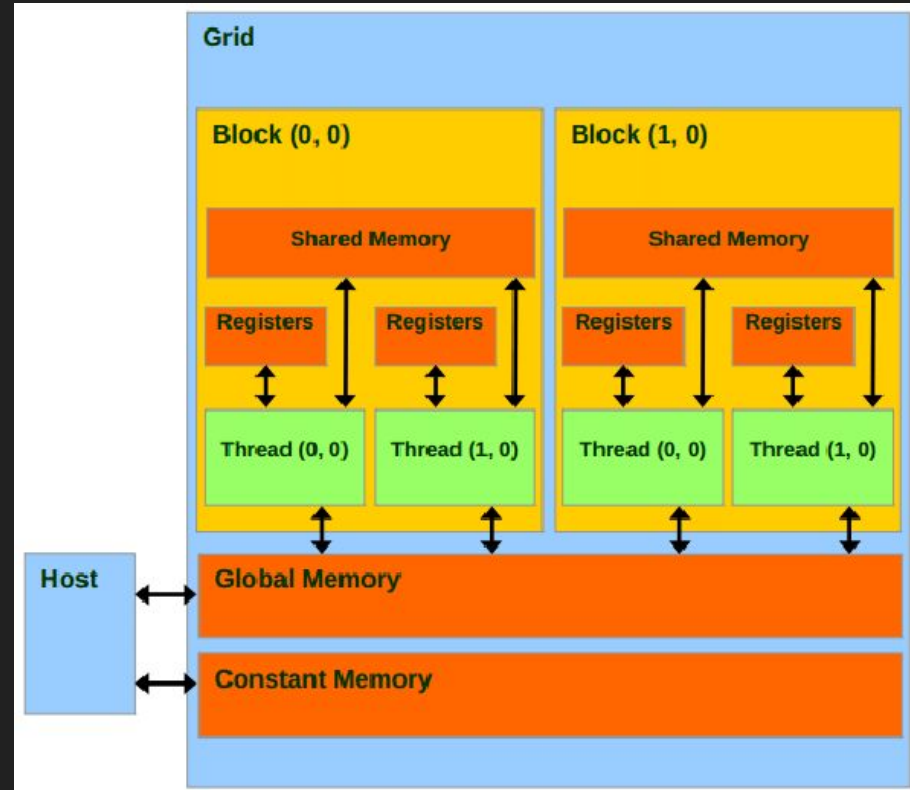
Cada SM tiene una caché de instrucciones, memoria constante, memoria de textura que es de solo lectura para el código del kernel



CUDA: Jerarquía de memoria

Memoria Global

Corresponde con la memoria destinada para el búfer de datos de la GPU y tiene el tamaño que especifica las características de la DRAM que se encuentran en la GPU



CUDA: Programación (compilador)

Compilación de un código CUDA:

- Compilador: Nvidia CUDA Compiler (**nvcc**)
- Extensión de los ficheros con código CUDA: **.cu**
- Compilación para código host y device independiente
- Admite flags de compilación: **-O2, -O3, Xptxas=-v -arch=sm11,...**

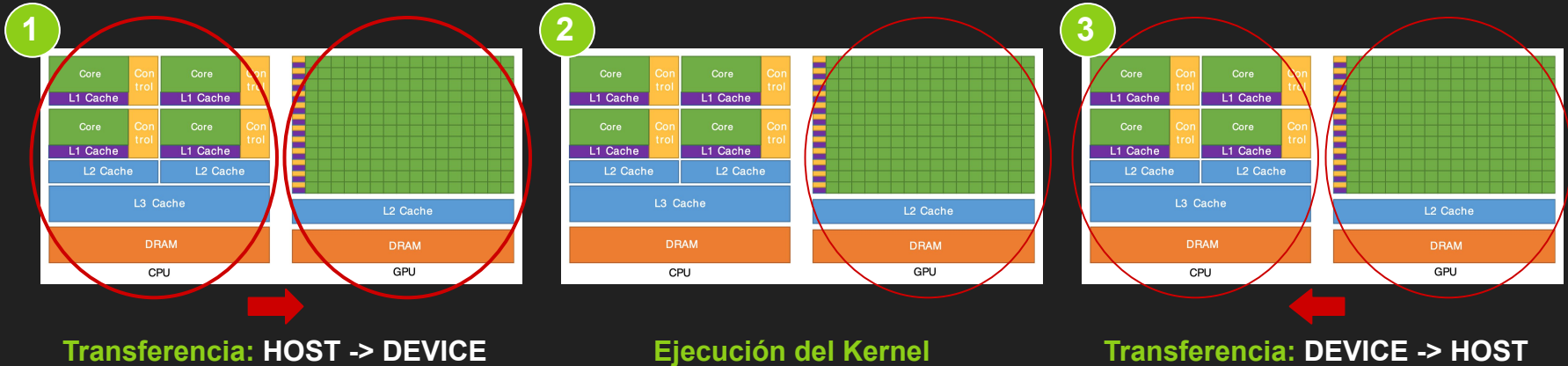
```
nvcc -O3 yourCode.cu -o yourBinary
```



CUDA: Programación (Flujo procesamiento)

Concepto

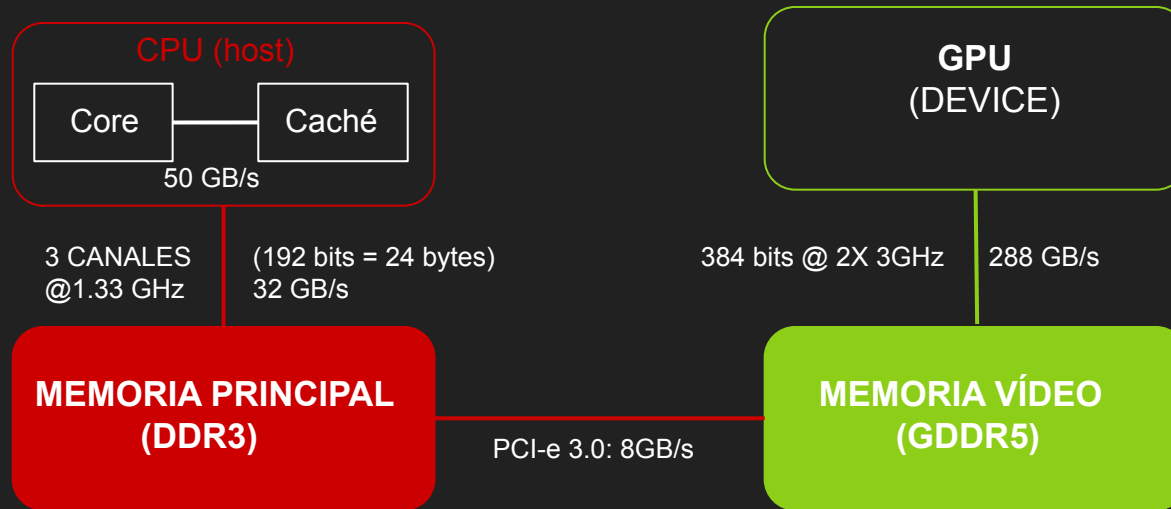
1. Mover datos de la memoria del HOST a la memoria del DEVICE
2. Lanzar el Kernel de ejecución desde el HOST en el DEVICE y procesar los datos en el DEVICE
3. Copiar los resultados desde el DEVICE al HOST



CUDA: Programación (Flujo procesamiento)

Limitaciones

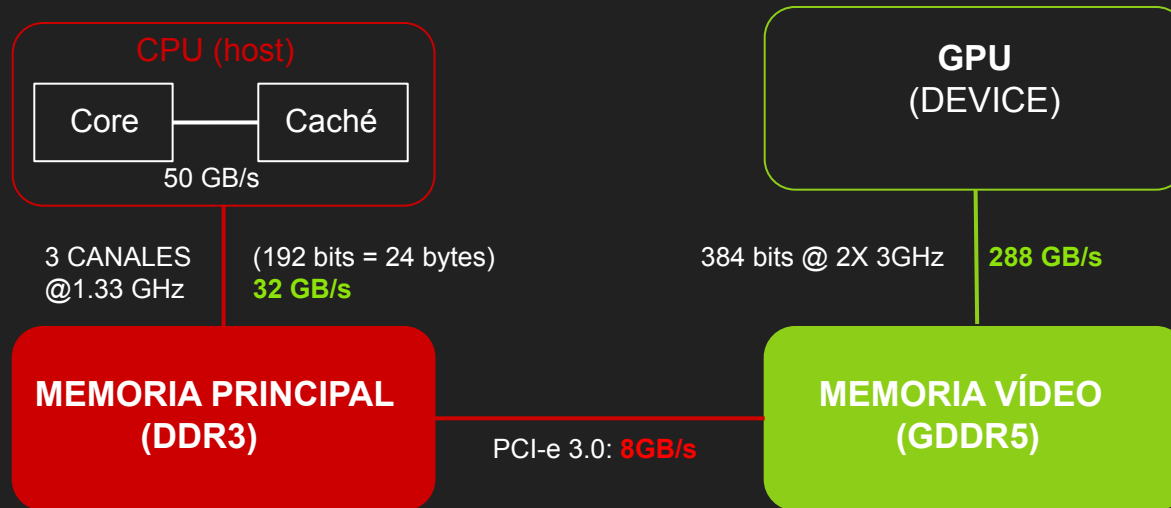
Evitar en la medida de lo posible realizar intercambios de memoria HOST<-> DEVICE



CUDA: Programación (Flujo procesamiento)

Limitaciones

Evitar en la medida de lo posible realizar intercambios de memoria HOST<-> DEVICE



CUDA: Programación (Tipos de funciones)

3 Tipos de definición de Funciones:

- `__device__`: Se ejecuta en el device y se llama desde el device
- `__global__`: Se ejecuta en el device y se llama desde el host
- `__host__`: Se ejecuta en el host y se llama desde el host

```
__global__ void Mi_kernel(int *d_a, int *d_b, int N)
```

Los Kernels usarán `__global__` y únicamente pueden hacer uso de C básico



Reserva de memoria:

- **cudaMalloc**: Reserva de memoria global en el DEVICE
- **cudaMemset**: Establece los bytes de una reserva a un valor determinado. Muy usado para inicializar a 0 arrays
- **cudaMemcpy**: Para realizar copias de memoria. Hay varias direcciones:
 - **cudaMemcpyHostToHost** : HOST -> HOST
 - **cudaMemcpyHostToDevice** : HOST -> DEVICE
 - **cudaMemcpyDeviceToHost** : DEVICE -> HOST
 - **cudaMemcpyDeviceToDevice** : DEVICE -> DEVICE
- **cudaFree**: Para liberar memoria

CUDA: Programación (Memoria)

- **cudaMalloc**(void** devPtr, size_t size)
int N = 16; //número de elementos del vector
int num_bytes = N*sizeof(int); // tamaño en bytes
int *d_a=0; //puntero para el device
cudaMalloc((void**)&d_a, num_bytes);
- **cudaMemset**(void* devPtr, int value, size_t count)
// establecer el valor 0 en todos los bytes del vector
cudaMemset(d_a, 0, num_bytes);
- **cudaMemcpy**(void* dst, const void* src, size_t count, cudaMemcpyKind kind)
// copia de los valores del vector del device al host
cudaMemcpy(h_a, d_a, num_bytes, cudaMemcpyDeviceToHost);
- **cudaFree**(void* devPtr)
// liberación de memoria utilizada
cudaFree(d_a);



CUDA: Programación (Ejemplo)

```
#include <stdio.h>
#include <cuda.h>
int main() {
    int N = 16; // número de elementos del vector
    int num_bytes = N*sizeof(int); // tamaño en bytes
    int *h_a=0; // puntero para el host
    int *d_a=0; // puntero para el device
    h_a = (int*) malloc(num_bytes); // reserva de memoria en el host para el vector
    cudaMalloc( (void**)&d_a, num_bytes); // reserva de memoria en el device para el vector
    if( h_a==0 || d_a==0 )
        printf("No pude reservar memoria\n");
    cudaMemset( d_a, 0, num_bytes); // establecer los bits del vector a 0
    // copia de los valores del vector del device al host
    cudaMemcpy( h_a, d_a, num_bytes, cudaMemcpyDeviceToHost);
    for (int i=0; i<N; i++)
        printf("%d ", h_a[i]);

    // liberación de memoria en el host y en el device
    free(h_a);
    cudaFree(d_a);
}
```



5 Palabras clave (no pueden usarse en la declaración de variables):

- **threadIdx**: Especifica el identificador de un hilo
- **blockIdx**: Especifica el identificador de un bloque
- **blockDim**: Especifica el número de hilos en un bloque
- **gridDim**: Especifica el número de bloques en un grid
- **dim3**: Tipo de variable tridimensional

- Los bloques y los grids pueden ser de 1D, 2D o 3D
- Cada una de las componentes dim3 representa cada una de las coordenadas de un eje cartesiano

```
dim3 blockSize(2,5); // 2 hilos en el eje x, 5 en el y  
dim3 gridSize(2,3); // 2 bloques en el eje x, 3 en el y
```

CUDA: Programación (Lanzamiento de kernels)

- Los Kernels son invocados para su ejecución desde el host de una forma similar a la que se llama a una función en C/C++
- Sintaxis: NombreKernel<<<gridSize, blockSize>>>(Parámetros del kernel)
- A no ser que se utilicen barreras de sincronización, la llamada a los kernels es asíncrona

```
int blockSize, gridSize;  
int n; // número de elementos en el vector  
// Número de hilos en cada bloque  
blockSize = 1024;  
  
// Número de bloques en el grid  
gridSize = (int)ceil((float)n/blockSize);  
  
// Ejecución del kernel  
add_kernel<<<gridSize, blockSize>>>(d_a, d_b, d_c, n);
```

CUDA: Programación (Número de bloques e hilos)

- Complicado de establecer un número óptimo inicial (heurística)
- Normalmente es deseable un número de hilos con un valor múltiplo de 21 (Warps)
- **blockSize**: Si la arquitectura de la GPU lo permite empezar por 256
- **gridSize**: El tamaño del grid debe cubrir por completo nuestro problema. Dividir el número de elementos “n” entre el número de hilos: $(int) \text{ceil} ((float)n / blockSize);$

```
int N = ...; //Número de hilos  
kernel<<<1, N>>>(A, B, C); //Llamada al kernel con N Hilos
```

```
dim3 threadsPerBlock(N, N);  
int numBlocks = 1;  
kernel<<<numBlocks, threadsPerBlock>>>(A, B, C); //Llamada al kernel con N * N * 1 Hilos
```

```
dim3 threadsPerBlock(16, 16);  
dim3 numBlocks( N / threadsPerBlock.x, N / threadsPerBlock.y);  
kernel<<<numBlocks, threadsPerBlock>>>(A, B, C);
```

<https://docs.nvidia.com/cuda/cuda-c-programming-guide/>

CUDA: Programación (Acceso a datos)

```
int n = 16; //vector de n elementos
dim3 blockSize(4); //4 bloques
dim3 gridSize(n/block.x); //4 hilos por bloque
kernel<<<gridSize, blockSize>>>(dA); //Llamada al kernel
```

```
__global__ void kernel( float *A ) {
    A[ blockIdx.x * blockDim.x + threadIdx.x ] = 7.0;
}
// Salida A[0...15]: 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7
__global__ void kernel( float *A ) {
    A[ blockIdx.x * blockDim.x + threadIdx.x ] = blockIdx.x;
}
__global__ void kernel( float *A ) {
    A[ blockIdx.x * blockDim.x + threadIdx.x ] = threadIdx.x;
}
__global__ void kernel( float *A ) {
    A[ blockIdx.x * blockDim.x + threadIdx.x ] = blockIdx.x * blockDim.x + threadIdx.x;
}
```



CUDA: Programación (Acceso a datos)

```
int n = 16; //vector de n elementos
dim3 blockSize(4); //4 bloques
dim3 gridSize(n/block.x); //4 hilos por bloque
kernel<<<gridSize, blockSize>>>(dA); //Llamada al kernel
```

```
__global__ void kernel( float *A ) {
    A[ blockIdx.x * blockDim.x + threadIdx.x ] = 7.0;
}
// Salida A[0...15]: 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7
__global__ void kernel( float *A ) {
    A[ blockIdx.x * blockDim.x + threadIdx.x ] = blockIdx.x;
}
// Salida A[0...15]: 0 0 0 0 1 1 1 1 2 2 2 2 3 3 3 3
__global__ void kernel( float *A ) {
    A[ blockIdx.x * blockDim.x + threadIdx.x ] = threadIdx.x;
}
__global__ void kernel( float *A ) {
    A[ blockIdx.x * blockDim.x + threadIdx.x ] = blockIdx.x * blockDim.x + threadIdx.x;
}
```



CUDA: Programación (Acceso a datos)

```
int n = 16; //vector de n elementos
dim3 blockSize(4); //4 bloques
dim3 gridSize(n/block.x); //4 hilos por bloque
kernel<<gridSize, blockSize>>>(dA); //Llamada al kernel
```

```
__global__ void kernel( float *A ) {
    A[ blockIdx.x * blockDim.x + threadIdx.x ] = 7.0;
}
// Salida A[0...15]: 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7

__global__ void kernel( float *A ) {
    A[ blockIdx.x * blockDim.x + threadIdx.x ] = blockIdx.x;
}
// Salida A[0...15]: 0 0 0 0 1 1 1 1 2 2 2 2 3 3 3 3

__global__ void kernel( float *A ) {
    A[ blockIdx.x * blockDim.x + threadIdx.x ] = threadIdx.x;
}
// Salida A[0...15]: 0 1 2 3 0 1 2 3 0 1 2 3 0 1 2 3

__global__ void kernel( float *A ) {
    A[ blockIdx.x * blockDim.x + threadIdx.x ] = blockIdx.x * blockDim.x + threadIdx.x;
}
```



CUDA: Programación (Acceso a datos)

```
int n = 16; //vector de n elementos
dim3 blockSize(4); //4 bloques
dim3 gridSize(n/block.x); //4 hilos por bloque
kernel<<gridSize, blockSize>>>(dA); //Llamada al kernel
```

```
__global__ void kernel( float *A ) {
    A[ blockIdx.x * blockDim.x + threadIdx.x ] = 7.0;
}
// Salida A[0...15]: 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7

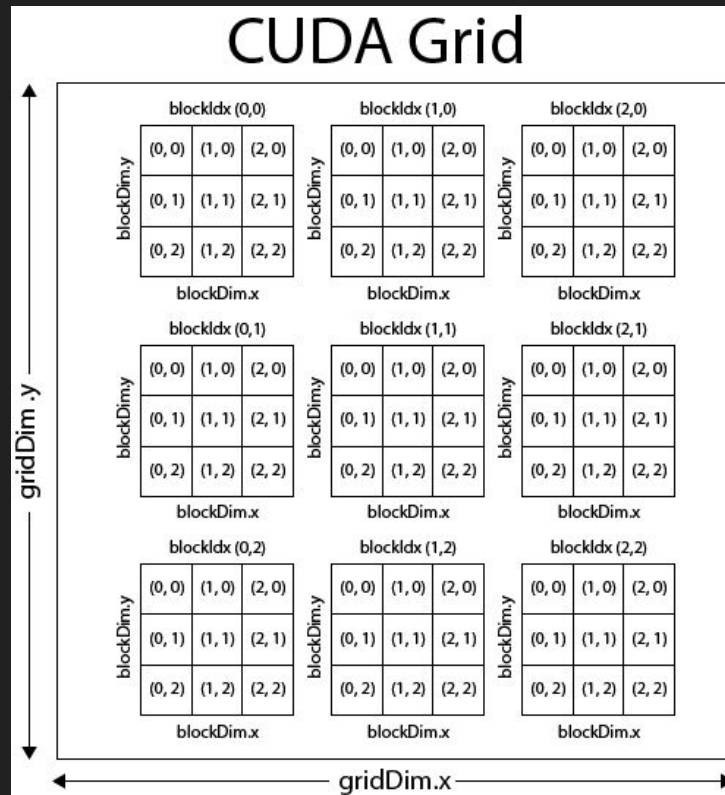
__global__ void kernel( float *A ) {
    A[ blockIdx.x * blockDim.x + threadIdx.x ] = blockIdx.x;
}
// Salida A[0...15]: 0 0 0 0 1 1 1 1 2 2 2 2 3 3 3 3

__global__ void kernel( float *A ) {
    A[ blockIdx.x * blockDim.x + threadIdx.x ] = threadIdx.x;
}
// Salida A[0...15]: 0 1 2 3 0 1 2 3 0 1 2 3 0 1 2 3

__global__ void kernel( float *A ) {
    A[ blockIdx.x * blockDim.x + threadIdx.x ] = blockIdx.x * blockDim.x + threadIdx.x;
}
// Salida A[0...15]: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
```



CUDA: Programación (Acceso a datos)



Fuente: 3dgep.com



Saxpy: $z = \alpha x + y$

Paralelizar con CUDA la implementación de la primera práctica (Tema 1)

Are You Doing All You Can?

